



Name:	Lyka Sophia S. Jardeleza	Date:	04/26/2025
Program:	BSCpE	Section:	A361

A. Guided Experiments:

(Please include the following: **1.** Photos and videos of the physical test outputs and **2.** Photos of the codes that was inputted in the Quartus Prime software.)

Experiment 1: Latch – D Latch in Verilog

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
Laboratory2_GuidedExercise1.mp4	<pre> 1 module d_latch(en, res, d, q); 2 3 input en, res, d; 4 output reg q; 5 6 always @ (*) 7 if (!res) 8 q <= 0; 9 else 10 if (en) 11 q <= d; 12 endmodule </pre>

Experiment 2: Clock – Blink LED

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
Laboratory2_GuidedExercise2.mp4	<pre> 1 module bLink_led(Clk, Q); 2 3 input Clk; 4 output Q; 5 6 reg [31:0] counter; 7 reg qr; 8 9 initial begin 10 counter <= 32'h00000000; 11 end 12 13 always @(posedge Clk) 14 begin 15 counter <= counter + 1; 16 17 if (counter == 32'd5000000) 18 qr <= 1; 19 else if (counter == 32'd10000000) //reset 20 begin 21 counter <= 32'h00000000; 22 qr <= 0; 23 end 24 end 25 26 assign Q = qr; 27 28 endmodule </pre>



Experiment 3: Flip-Flops – JK Flip-Flop in Verilog

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
Laboratory2_GuidedExercise3.mp4	<pre> 1 module jk_fflop (clk, j, k, q, out_clk); 2 3 input clk, j, k; 4 output q, out_clk; 5 6 set_clk _clk(.clk(clk), .q(out_clk)); //timer 7 jk_ff _jk_ff(.clk(out_clk), .j(~j), .k(~k), .q(q)); //flip-flop 8 9 endmodule 10 11 12 module set_clk(clk, Q); 13 14 input clk; 15 output Q; 16 17 reg [31:0] counter; 18 reg qr; 19 20 initial begin 21 counter <= 32'h00000000; 22 end 23 24 always @(posedge clk) 25 begin 26 counter <= counter + 1; 27 28 if (counter == 32'd250000000) //1 Hz 29 qr <= 1; 30 else if (counter == 32'd500000000) //reset 31 begin 32 counter <= 32'h00000000; 33 qr <= 0; 34 end 35 end 36 assign Q = qr; 37 endmodule 38 39 40 module jk_ff (clk, J, K, Q); 41 42 input clk, J, K; 43 output Q; 44 45 reg q; 46 47 always @ (posedge clk) 48 case ({J, K}) 49 2'b00 : q <= q; 50 2'b01 : q <= 0; 51 2'b10 : q <= 1; 52 2'b11 : q <= ~q; 53 endcase 54 assign Q = q; 55 endmodule 56 57 58 </pre>

Experiment 4: Flip-Flops – T Flip-Flop in Verilog

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
Laboratory2_GuidedExercise4.mp4	<pre> 1 module t_fflop (clk, t, q, out_clk); 2 3 input clk, t; 4 output q, out_clk; 5 6 set_clk _clk(.clk(clk), .q(out_clk)); //timer 7 jk_ff _jk_ff(.clk(out_clk), .j(~t), .k(~t), .q(q)); //flip-flop 8 9 endmodule 10 11 12 module set_clk(clk, Q); 13 14 input clk; 15 output Q; 16 17 reg [31:0] counter; 18 reg qr; 19 20 initial begin 21 counter <= 32'h00000000; 22 end 23 24 always @(posedge clk) 25 begin 26 counter <= counter + 1; 27 28 if (counter == 32'd250000000) //1 Hz 29 qr <= 1; 30 else if (counter == 32'd500000000) //reset 31 begin 32 counter <= 32'h00000000; 33 qr <= 0; 34 end 35 end 36 assign Q = qr; 37 endmodule 38 39 40 module jk_ff (clk, J, K, Q); 41 42 input clk, J, K; 43 output Q; 44 45 reg q; 46 47 always @ (posedge clk) 48 case ({J, K}) 49 2'b00 : q <= q; 50 2'b01 : q <= 0; 51 2'b10 : q <= 1; 52 2'b11 : q <= ~q; 53 endcase 54 assign Q = q; 55 endmodule 56 57 58 </pre>



Experiment 5: Counter – Ripple Counter in Verilog

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
Laboratory2_GuidedExercise5.mp4	<pre> 1 module ff_counter(clk, out); 2 3 input clk; 4 output [3:0] out; 5 6 wire wr_clk; 7 8 set_clk_c1k(clk, .Q(wr_clk)); //timer 9 10 //ripple counter 11 jk_ff jk_ff0(.clk(~wr_clk), .J(1), .K(1), .Q(out[0])); //ff_0 12 jk_ff jk_ff1(.clk(~out[0]), .J(1), .K(1), .Q(out[1])); //ff_1 13 jk_ff jk_ff2(.clk(~out[1]), .J(1), .K(1), .Q(out[2])); //ff_2 14 jk_ff jk_ff3(.clk(~out[2]), .J(1), .K(1), .Q(out[3])); //ff_3 15 16 17 endmodule 18 19 module set_clk(clk, Q); 20 21 input clk; 22 output Q; 23 24 reg [31:0] counter; 25 reg qr; 26 27 initial begin 28 counter <= 32'h00000000; 29 end 30 31 always @(posedge clk) 32 begin 33 counter <= counter + 1; 34 35 if (counter == 32'd25000000) //1 Hz 36 qr <= 1; 37 else if (counter == 32'd50000000) //reset 38 begin 39 counter <= 32'h00000000; 40 qr <= 0; 41 end 42 end 43 assign Q = qr; 44 endmodule 45 46 module jk_ff (clk, J, K, Q); 47 48 input clk, J, K; 49 output Q; 50 51 reg q; 52 53 always @(posedge clk) 54 case ({J, K}) 55 2'b00 : q <= q; 56 2'b01 : q <= 0; 57 2'b10 : q <= 1; 58 2'b11 : q <= ~q; 59 endcase 60 assign Q = q; 61 endmodule 62 63 64 65 66 </pre>



B. Coding Exercise:

(Please include the following: **1. Photos and videos of the physical test outputs**, **2. Photos of the codes** that was inputted in the Quartus Prime software, and **3. The raw source code** that you made for the coding exercise.)

Coding Exercise 1: Ripple Counter to 7-Segment Decimal

1. Photos of Physical Test Outputs	2. Photos of Codes
Laboratory2_CodingExercise1.mp4	<pre> 1 module ripple_to_svn (input clk, output [6:0] seg_a, output [6:0] seg_b); 2 3 wire wr_clk; 4 wire [3:0] count; 5 6 // Generate slow clock (0.5 Hz for 2-second interval) 7 set_clk_clk (.Clk(clk), .Q(wr_clk)); 8 9 // 4-bit ripple counter 10 wire q0, q1, q2, q3; 11 jk_ff jk0 (.Clk(~wr_clk), .J(1'b1), .K(1'b1), .Q(q0)); 12 jk_ff jk1 (.Clk(~q0), .J(1'b1), .K(1'b1), .Q(q1)); 13 jk_ff jk2 (.Clk(~q1), .J(1'b1), .K(1'b1), .Q(q2)); 14 jk_ff jk3 (.Clk(~q2), .J(1'b1), .K(1'b1), .Q(q3)); 15 16 assign count = {q3, q2, q1, q0}; 17 18 // 7-segment display driver 19 seven_segment_decoder decoder (20 .binary_in(count), 21 .seg_a(seg_a), 22 .seg_b(seg_b) 23); 24 25 endmodule 26 27 28 module set_clk(input clk, output Q); 29 reg [21:0] counter = 0; 30 reg qr = 0; 31 32 always @(posedge clk) begin 33 counter <= counter + 1; 34 if (counter == 32'd50000000) begin // 2 seconds for 50 MHz clock 35 qr <= ~qr; 36 counter <= 0; 37 end 38 end 39 40 assign Q = qr; 41 endmodule 42 43 44 module jk_ff(input clk, input J, input K, output Q); 45 reg q = 0; 46 47 always @(posedge clk) begin 48 if (J == 1 && K == 1) 49 q <= ~q; 50 else 51 q <= q; 52 end 53 54 assign Q = q; 55 endmodule 56 57 58 module seven_segment_decoder(input [3:0] binary_in, output [6:0] seg_a, output [6:0] seg_b); 59 reg [6:0] seg0, seg1; 60 61 always @(*) begin 62 case (binary_in < 10) 63 0: seg0 = "b1000000; 64 1: seg0 = "b1111001; 65 2: seg0 = "b01100100; 66 3: seg0 = "b0110000; 67 4: seg0 = "b0011001; 68 5: seg0 = "b0110010; 69 6: seg0 = "b0000010; 70 7: seg0 = "b1111000; 71 8: seg0 = "b0000000; 72 9: seg0 = "b0011000; 73 default: seg0 = "b1111111; 74 endcase 75 76 case (binary_in < 10) 77 0: seg1 = "b1000000; 78 1: seg1 = "b1111001; 79 default: seg1 = "b1111111; 80 endcase 81 end 82 83 assign seg_a = seg1; 84 assign seg_b = seg0; 85 endmodule 86 </pre>

Raw Source Code:

```

module ripple_to_svn (input clk, output [6:0] seg_a, output [6:0] seg_b);

    wire wr_clk;
    wire [3:0] count;

    // Generate slow clock (0.5 Hz for 2-second interval)
    set_clk_clk (.Clk(clk), .Q(wr_clk));

    // 4-bit ripple counter
    wire q0, q1, q2, q3;
    jk_ff jk0 (.Clk(~wr_clk), .J(1'b1), .K(1'b1), .Q(q0));
    jk_ff jk1 (.Clk(~q0), .J(1'b1), .K(1'b1), .Q(q1));
    jk_ff jk2 (.Clk(~q1), .J(1'b1), .K(1'b1), .Q(q2));
    jk_ff jk3 (.Clk(~q2), .J(1'b1), .K(1'b1), .Q(q3));

    assign count = {q3, q2, q1, q0};

    // 7-segment display driver
    seven_segment_decoder decoder (
        .binary_in(count),

```



```
.seg_a(seg_a),
.seg_b(seg_b)
);

endmodule

module set_clk(input Clk, output Q);
    reg [31:0] counter = 0;
    reg qr = 0;

    always @(posedge Clk) begin
        counter <= counter + 1;
        if (counter == 32'd50000000) begin // 2 seconds for 50 MHz clock
            qr <= ~qr;
            counter <= 0;
        end
    end

    assign Q = qr;
endmodule

module jk_ff(input Clk, input J, input K, output Q);
    reg q = 0;

    always @(posedge Clk) begin
        if (J == 1 && K == 1)
            q <= ~q;
        end

    assign Q = q;
endmodule

module seven_segment_decoder(input [3:0] binary_in, output [6:0] seg_a, output [6:0] seg_b);
    reg [6:0] seg0, seg1;

    always @(*) begin
        case (binary_in % 10)
            0: seg0 = 7'b1000000;
            1: seg0 = 7'b1111001;
            2: seg0 = 7'b0100100;
            3: seg0 = 7'b0110000;
            4: seg0 = 7'b0011001;
            5: seg0 = 7'b0010010;
            6: seg0 = 7'b0000010;
            7: seg0 = 7'b1111000;
            8: seg0 = 7'b0000000;
            9: seg0 = 7'b0011000;
            default: seg0 = 7'b1111111;
        endcase

        case (binary_in / 10)
            0: seg1 = 7'b1000000;
            1: seg1 = 7'b1111001;
            default: seg1 = 7'b1111111;
        endcase
    end

    assign seg_a = seg1;
    assign seg_b = seg0;
endmodule
```



Coding Exercise 2: CPE112L Looping Message

1. Photos of Physical Test Outputs	2. Photos of Codes
Laboratory2_CodingExercise2.mp4	<pre> 1 module looping_message (2 input clk, 3 output [6:0] seg0, seg1, seg2, seg3, seg4, seg5 4); 5 6 wire slow_clk; 7 reg [2:0] seq = 0; // 0 to 6 8 reg [6:0] displays [5:0]; 9 10 // Letter to 7-segment mappings (inside the module) 11 localparam [6:0] C = 7'b1000110; // C 12 localparam [6:0] P = 7'b0001100; // P 13 localparam [6:0] E = 7'b0000110; // E 14 localparam [6:0] ONE = 7'b1111001; // 1 15 localparam [6:0] TWO = 7'b0100100; // 2 16 localparam [6:0] L = 7'b1000111; // L 17 localparam [6:0] BLANK = 7'b1111111; // blank 18 19 // Slow down the clock to 800ms 20 clock_divider800ms clkdiv (.clk(clk), .slow_clk(slow_clk)); 21 22 always @(posedge slow_clk) begin 23 seq <= (seq == 3'd6) ? 0 : seq + 1; 24 end 25 26 always @(*) begin 27 case(seq) 28 0: begin displays[5]=C; 29 displays[4]=P; 30 displays[3]=E; 31 displays[2]=ONE; 32 displays[1]=ONE; 33 displays[0]=TWO; 34 end 35 1: begin displays[5]=P; 36 displays[4]=E; 37 displays[3]=ONE; 38 displays[2]=ONE; 39 displays[1]=TWO; 40 displays[0]=L; 41 end 42 2: begin displays[5]=E; 43 displays[4]=ONE; 44 displays[3]=ONE; 45 displays[2]=TWO; 46 displays[1]=L; 47 displays[0]=C; 48 end 49 3: begin displays[5]=ONE; 50 displays[4]=ONE; 51 displays[3]=TWO; 52 displays[2]=L; 53 displays[1]=C; 54 displays[0]=P; 55 end 56 4: begin displays[5]=ONE; 57 displays[4]=TWO; 58 displays[3]=L; 59 displays[2]=C; 60 displays[1]=P; 61 displays[0]=E; 62 end 63 5: begin displays[5]=TWO; 64 displays[4]=L; 65 displays[3]=C; 66 displays[2]=P; 67 displays[1]=E; 68 displays[0]=ONE; 69 end 70 6: begin displays[5]=L; 71 displays[4]=P; 72 displays[3]=P; 73 displays[2]=E; 74 displays[1]=ONE; 75 displays[0]=ONE; 76 end 77 default: begin displays[5]=BLANK; 78 displays[4]=BLANK; 79 displays[3]=BLANK; 80 displays[2]=BLANK; 81 displays[1]=BLANK; 82 displays[0]=BLANK; 83 end 84 endcase 85 end 86 87 assign seg0 = displays[0]; 88 assign seg1 = displays[1]; 89 assign seg2 = displays[2]; 90 assign seg3 = displays[3]; 91 assign seg4 = displays[4]; 92 assign seg5 = displays[5]; 93 94 endmodule 95 96 // Module to generate slow clock every 800ms 97 module clock_divider800ms(input clk, output reg slow_clk); 98 reg [31:0] counter = 0; 99 100 always @(posedge clk) begin 101 if (counter == 32'd4000000) begin // 800ms for 50MHz clock 102 slow_clk <= ~slow_clk; 103 counter <= 0; 104 end 105 else 106 counter <= counter + 1; 107 end 108 endmodule </pre>

Raw Source Code:

```
module looping_message (
    input clk,
    output [6:0] seg0, seg1, seg2, seg3, seg4, seg5
);

    wire slow_clk;
    reg [2:0] seq = 0; // 0 to 6
    reg [6:0] displays [5:0];

    // Letter to 7-segment mappings (inside the module)
    localparam [6:0] C    = 7'b1000110; // C
    localparam [6:0] P    = 7'b0001100; // P
    localparam [6:0] E    = 7'b0000110; // E
    localparam [6:0] ONE  = 7'b1111001; // 1
    localparam [6:0] TWO  = 7'b0100100; // 2
    localparam [6:0] L    = 7'b1000111; // L
    localparam [6:0] BLANK = 7'b1111111; // blank

    // Slow down the clock to 800ms
    clock_divider800ms clkdiv (.clk(clk), .slow_clk(slow_clk));

    always @(posedge slow_clk) begin
        seq <= (seq == 3'd6) ? 0 : seq + 1;
    end

    always @(*) begin
        case(seq)
            0: begin displays[5]=C;

                                displays[4]=P;
                                displays[3]=E;
                                displays[2]=ONE;
                                displays[1]=ONE;
                                displays[0]=TWO;

                                end
            1: begin displays[5]=P;

                                displays[4]=E;
                                displays[3]=ONE;
                                displays[2]=ONE;
                                displays[1]=TWO;
                                displays[0]=L;

                                end
            2: begin displays[5]=E;

                                displays[4]=ONE;
                                displays[3]=ONE;
                                displays[2]=TWO;
                                displays[1]=L;
                                displays[0]=C;

                                end
            3: begin displays[5]=ONE;

                                displays[4]=ONE;
                                displays[3]=TWO;
                                displays[2]=L;
                                displays[1]=C;
                                displays[0]=P;

                                end
            4: begin displays[5]=ONE;

                                displays[4]=TWO;
                                displays[3]=L;
                                displays[2]=C;
                                displays[1]=P;
                                displays[0]=E;

                                end
            5: begin displays[5]=TWO;

                                displays[4]=L;
                                displays[3]=C;

                                end
        endcase
    end
```



```

                                displays[2]=P;
                                displays[1]=E;
                                displays[0]=ONE;

                                end
6: begin displays[5]=L;

                                displays[4]=C;
                                displays[3]=P;
                                displays[2]=E;
                                displays[1]=ONE;
                                displays[0]=ONE;

                                end
default: begin displays[5]=BLANK;

                                displays[4]=BLANK;
                                displays[3]=BLANK;
                                displays[2]=BLANK;
                                displays[1]=BLANK;
                                displays[0]=BLANK;

                                end

                                endcase
                                end

                                assign seg0 = displays[0];
                                assign seg1 = displays[1];
                                assign seg2 = displays[2];
                                assign seg3 = displays[3];
                                assign seg4 = displays[4];
                                assign seg5 = displays[5];

                                endmodule

// Module to generate slow clock every 800ms
module clock_divider800ms(input clk, output reg slow_clk);
    reg [31:0] counter = 0;

    always @(posedge clk) begin
        if (counter == 32'd40000000) begin // 800ms for 50MHz clock
            slow_clk <= ~slow_clk;
            counter <= 0;
        end
        else
            counter <= counter + 1;
        end
    end
endmodule
```




C. Findings ,Observations, and Comments:

Guiding Questions :

1. *What did you learn in the guided experiments? Did you learn something new?*
2. *What are your findings, observations, and comments in the guided experiments?*
3. *Did you solve the coding exercise?*
4. *Are there any difficulties during coding or set-up that you encountered?*
5. *What are your findings, observations, and comments in the coding exercise?*

Through the guided experiments, I learned how to implement different digital components like latches, flip-flops, and counters in Verilog. I gained a deeper understanding of how sequential logic works, especially in controlling the flow of data through these components. The concept of ripple counters was particularly interesting, as it allowed me to see how multiple flip-flops can be used together to generate a counting sequence. One key observation is how different flip-flops (JK and T) can be used in similar ways to create counters or memory elements in digital circuits. Additionally, the importance of clock management was evident throughout the experiments, particularly when working with the slow clock module to manage timing in the ripple counter and the looping message display. Yes, I was able to successfully solve both coding exercises. The first exercise, the Ripple Counter to 7-segment decimal display, worked as expected, and I was able to visualize the counting sequence on the 7-segment displays. The second exercise, the looping message on 7-segment displays, also worked well. One difficulty I encountered was ensuring that the clock divider was functioning correctly, especially when implementing the slow clock for both the ripple counter and the looping message. The timing had to be precise, and I had to make sure the clock frequency was slowed down to a manageable rate for the displays. Additionally, understanding how to handle the multiple 7-segment displays in the second exercise took some trial and error. In the coding exercise, I found that managing multiple 7-segment displays required careful attention to the timing and the correct mapping of values to each display. In the looping message exercise, the use of a sequence and the slow clock to control the timing of the message was a good approach to display text in a repeating pattern. Overall, both exercises were useful in reinforcing the concepts of timing, sequential logic, and display management in digital systems.